

```

num_total_tickets += num_tickets_per_thread[i]*number;
printf ("%d", num_total_tickets);
}

```

As our readers correctly pointed out, in order to avoid false sharing, each element of the array `num_tickets_per_thread` needs to be placed on a separate cache line. The modified code given below does not exhibit any false sharing. We add a pad of size 60 bytes (assuming a cache line size of 64 bytes) around each element of the array so that each element is located on a different cache line. Hence, updating of one element of the array does not result in invalidating copies of any other element present in caches of other processors.

Issues to be aware of in lock-based programming

We'll now look at some of the issues associated with multi-threaded programming that arise due to the use of locks and how they can be overcome. Locks are used to protect the access to shared resources in a multi-threaded application. While locks are needed to ensure a consistent application state, they also bring forth certain issues. We have already discussed a couple of these, namely deadlock and data races. In today's column, we will look at another potential problem associated with locks, called *priority inversion*.

Consider an application that has two threads -- a higher priority thread T1 and a lower priority thread T2. Both T1 and T2 need to update a shared global counter that counts the number of operations performed in the application. This shared global counter is protected by a lock, L1. Now when the higher priority thread T1 is ready to run, it ends up waiting for the lower priority thread T2, since T2 currently holds the lock L1. Thus even if T1 has a higher priority than T2, T1 ends up waiting for T2 to finish updating the counter and hence release the lock L1, before it can run. Such a situation is known as priority inversion since T1 has a higher priority than T2, but ends up waiting for T2 clearly resulting in a priority inversion. In an application with a large number of threads, this situation can result in the higher priority thread waiting for a long time on lower priority threads because of access to common shared resources.

A famous case of priority inversion

In many cases, priority inversion happens without the programmer being aware of it and it does not have a major impact on systems performance. But there have been cases where priority inversion has caused poor response on performance, especially in real-time systems. A famous case of priority inversion occurred in the software used in the Mars Pathfinder mission in July 1997. The Pathfinder mission took high-resolution colour pictures of the Martian surface and relayed them back to Earth. When the software was employed on

Mars, the programmers back on earth were puzzled by the number of software resets that occurred frequently, leading to poor systems performance. On investigation, it was found that the issue was due to priority inversion.

In the Martian spacecraft, various devices communicated over a data bus. Activity on this bus was managed by a pair of high-priority tasking threads -- BM1 and BM2. One of the bus manager tasks that BM1 communicated through a pipe was a low-priority meteorological science task, MS1. The communication pipe was protected by a mutex L1. BM1 and MS1 needed to acquire the mutex L1 before they could send data over the pipe.

The sequence of events leading to each reset began when the low-priority task MS1 was pre-empted by a couple of medium-priority tasks while it held the mutex L1. While the low-priority task MS1 was pre-empted, the high-priority bus distribution manager thread, BM1, tried to send more data to it over the same pipe. Because the mutex L1 was still held by MS1, the bus distribution manager was made to wait. Shortly thereafter, the other bus scheduler thread BM2 became active. It noticed that the distribution manager BM1 hadn't completed its work for that bus cycle and forced a system reset. VxWorks was the RTOS used on Pathfinder and using the priority inversion workaround VxWorks had, NASA engineers were able to solve the issue remotely. More details on the priority inversion issue encountered in the Pathfinder mission, can be found in David Wilner's keynote talk at the 1997 IEEE real-time symposium: http://www.research.microsoft.com/research/os/mjb/Mars_Pathfinder/Mars_Pathfinder.html

Dealing with priority inversion

One way of dealing with priority inversion is to use a technique known as priority inheritance. With priority inheritance being enforced by the operating system, a lower priority task will inherit the task of any other higher priority task that is waiting on a shared resource currently owned by the lower priority task. Consider a simple example of two tasks T1 and T2 with priorities P1 and P2, respectively, with P1 being higher than P2. Hence T1 is the higher priority task compared to T2. Both access a shared resource protected by a mutex L1.

Assume that T2 holds the mutex L1 currently. Now, if T1 becomes ready and needs to wait for mutex L1, then T2's priority is boosted to P1. This is done so that T2 can finish quickly and release the mutex so that the higher priority task T1 can then run. So T2 temporarily gets a higher priority for a short time. Since T2 inherits the priority from T1, this technique is known as priority inheritance. Priority change due to priority inheritance takes place as soon as the high-priority task begins to be pending; it ends when the resource is released by the lower priority thread.